

InsightClass 项目架构文档

本文档面向所有团队成员，即使没有编程经验也能看懂。
最后更新：2026-06-03

一、项目整体概览

这个项目是干什么的？

InsightClass 是一个**课堂学生行为检测系统**。简单来说，它能从前排摄像头拍到的教室画面中，自动识别出四种行为：

行为 ID	中文名	场景举例
phone_use	玩手机	学生低头看手机
talking	交谈	学生之间交头接耳
sleeping	打瞌睡	学生趴桌子或低头闭眼
standing	站立	学生站起来回答问题等

整个项目分成哪几个阶段？

录制视频 → 抽帧 → 标注 → 训练模型 → 推理检测 → 可视化/Web 展示

你可以把它想象成"教电脑认人"的过程：

1. **拍视频**：用教室里的摄像头录制上课画面
2. **抽帧**：从视频里每隔几秒截一张图
3. **标注**：人工在图片上框出"这里有人在玩手机"、"这里有人在打瞌睡"
4. **训练**：把这些标注好的图片喂给 AI 模型，让它学会识别
5. **推理**：训练好的模型看到新图片就能自动识别
6. **展示**：把识别结果用网页或视频叠加的方式展示出来

技术栈（用了哪些工具？）

层次	技术	作用
编程语言	Python 3.10+	整个项目的基础
AI 模型	Ultralytics YOLO (YOLO11n / YOLO26n)	目标检测的核心引擎
图像处理	OpenCV (cv2)	读取视频、抽帧、图像处理
数据格式	PyYAML、JSON、CSV	配置文件和数据存储
Web 框架	FastAPI + Jinja2 + uvicorn	网页端推理界面
数据验证	Pydantic	API 请求/响应的数据结构校验
可视化	supervision、PIL/Pillow	在图片/视频上画检测框
包管理	pip + setuptools (src-layout)	安装和分发
版本控制	Git + Gitee	代码托管

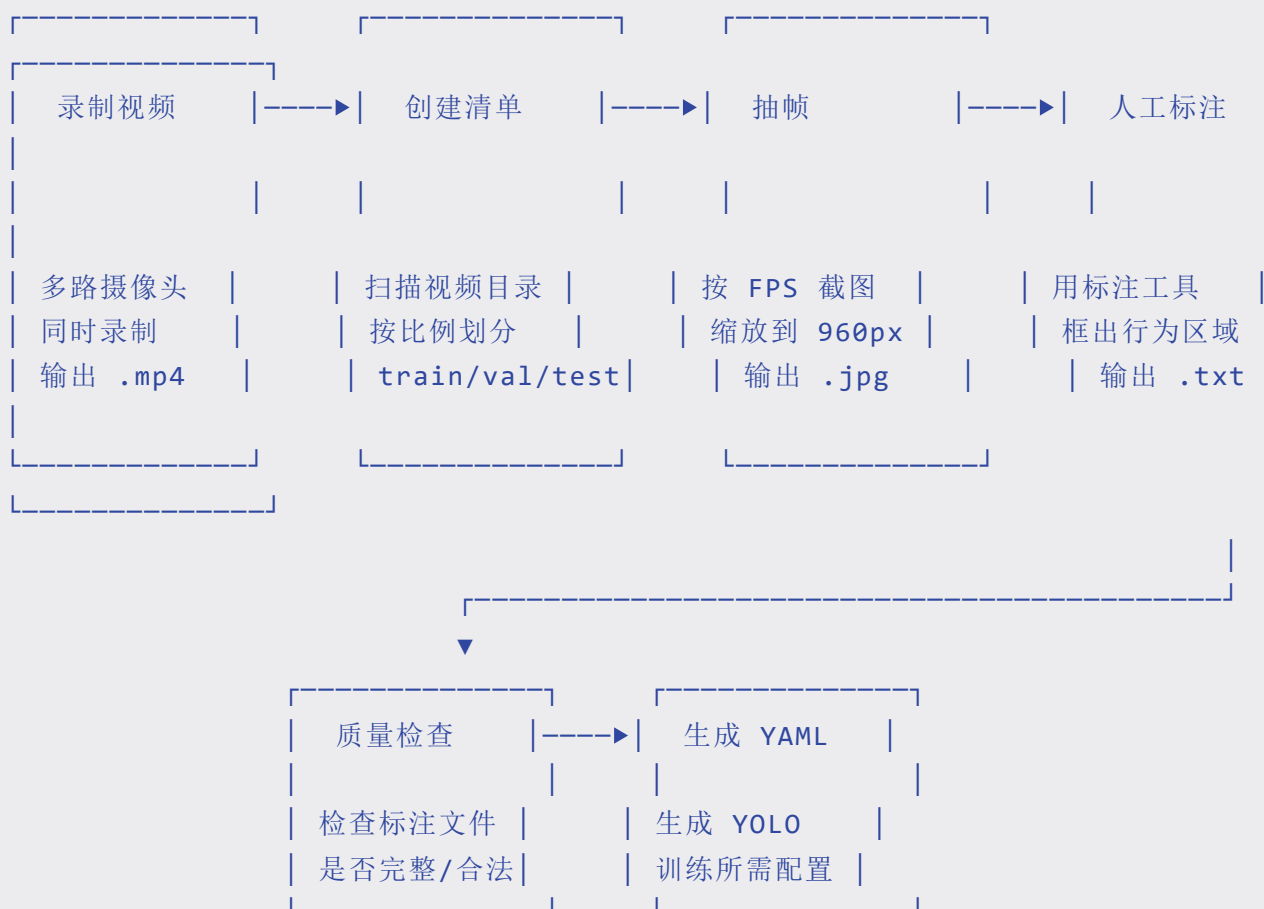
二、核心目录结构

InsightClass/	# 项目根目录
-- CLAUDE.md	# AI 助手指引文件（给 Claude Code 用的）
-- README.md	# 项目简介
-- LICENSE	# MIT 开源协议
-- pyproject.toml	# 项目配置：依赖、安装方式、入口点
-- test_environment.py	# 环境检查脚本（检查 CUDA、PyTorch 等是否装好）
-- configs/	# ★ 所有 YAML 配置文件
-- classes.yaml	# 类别定义（4 种行为的 ID 和中文名）
-- cameras.yaml	# 自定义摄像头列表（运行时生成）


```
└-- {实验名}/                                # 每次训练一个子目录
  |-- weights/best.pt                        # 最佳模型权重
  |-- weights/last.pt                       # 最后一轮权重
  |-- results.csv                           # 训练指标记录
  |-- confusion_matrix.png                  # 混淆矩阵图
  └-- experiment_record.json                 # 实验记录（超参+指标+路径）
```

三、核心业务流程

3.1 数据处理全流程（从视频到可训练的数据集）



对应命令：

```
# 第 1 步：创建数据集清单（划分 train/val/test）
insightclass create-manifest --config configs/dataset_manifest.example.yaml
--output data/.../manifest.yaml

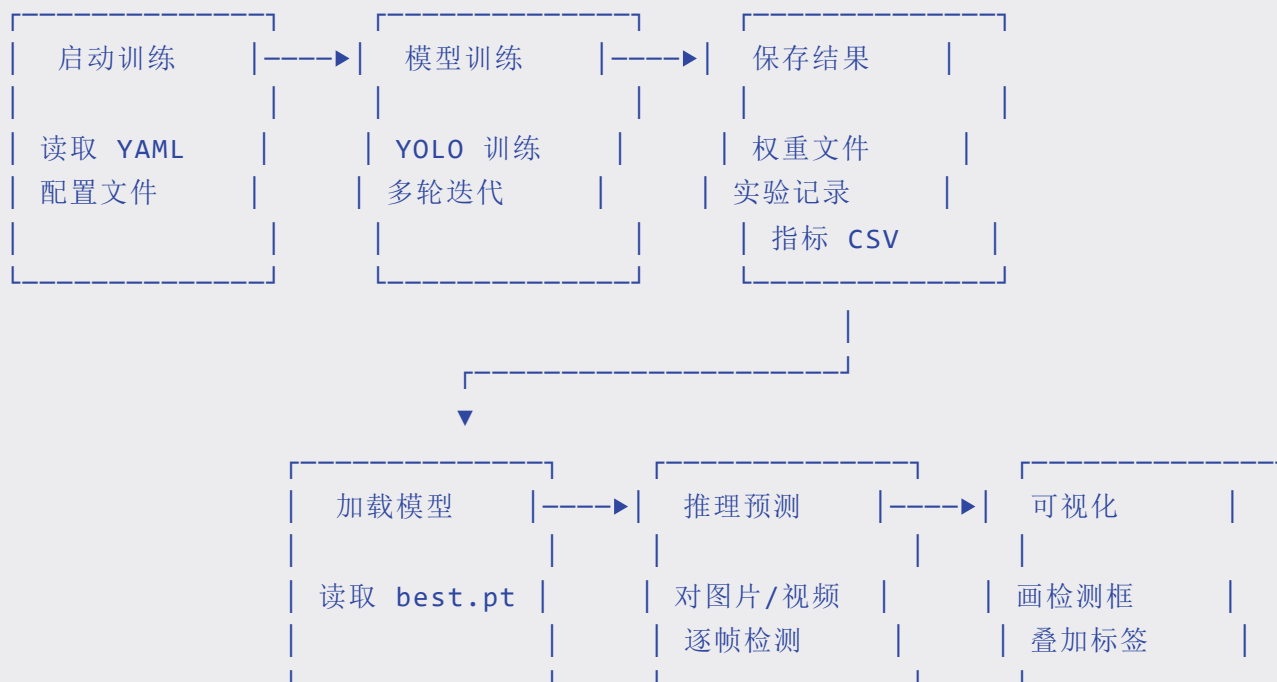
# 第 2 步：从视频中抽帧
insightclass extract-frames --manifest <manifest路径> --fps 1.0 --target-
width 960

# 第 3 步：人工标注（用 X-AnyLabeling 等外部工具，非命令行）

# 第 4 步：检查标注质量
insightclass inspect-yolo --dataset-root <数据集根目录> --class-config
configs/classes.yaml --output reports/...

# 第 5 步：生成训练配置文件
insightclass write-yolo-yaml --dataset-root <数据集根目录> --class-config
configs/classes.yaml --output <输出路径>
```

3.2 训练与推理流程



对应命令：

```
# 训练
insightclass train --config configs/training_v2_ultralytics.yaml
```

```

# 验证
insightclass validate --config <同上>

# 推理
insightclass predict --config configs/inference.ultralytics.example.yaml

# 可视化第一帧
insightclass render-first-frame --config <同上>

# 对比多次实验结果
insightclass compare-experiments --experiments-root experiments --output
reports/experiment_summary.csv

# 查看实验结果（启动 Web 界面）
insightclass view-experiments --experiments-root experiments --port 8001

# 启动统一 Demo（推理 + 实验查看）
insightclass demo --experiments-root experiments --port 8000

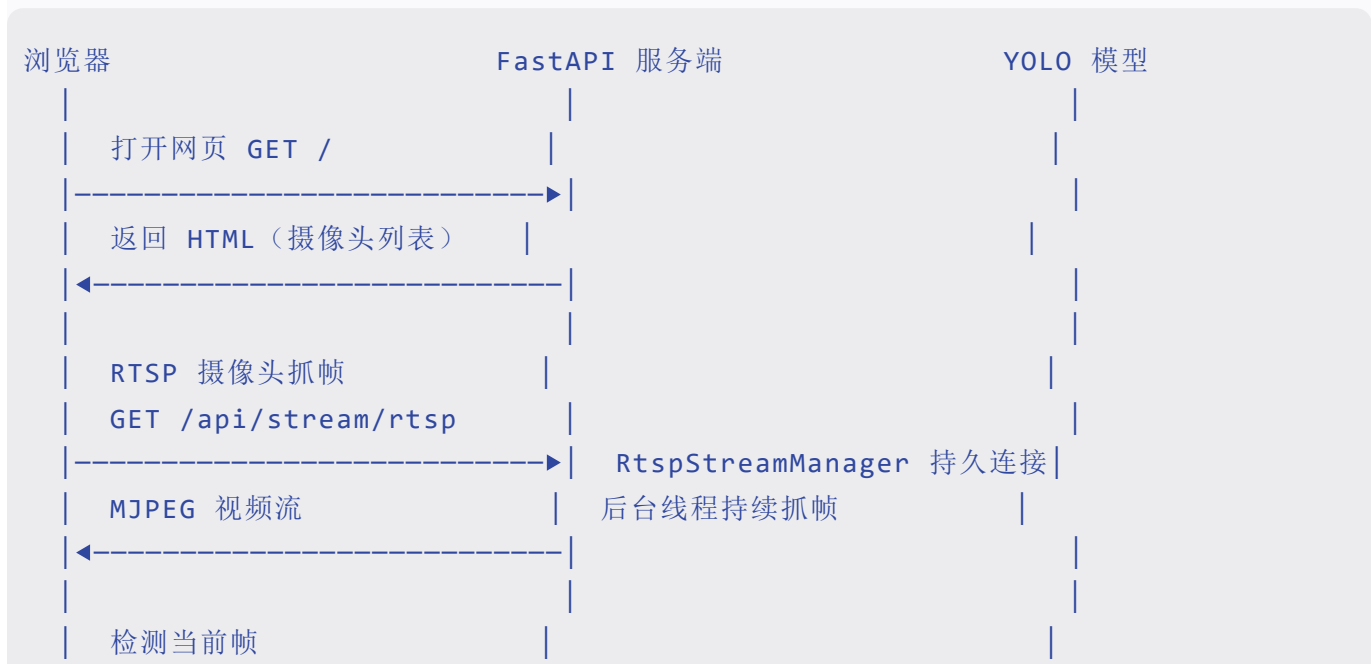
```

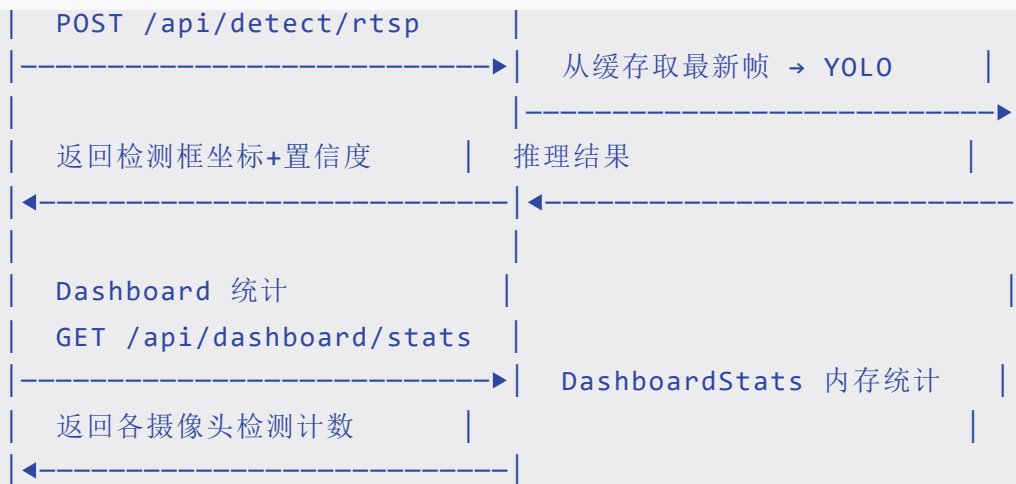
3.3 Web 服务流程

系统提供三个独立的 Web 应用，各有不同用途：

主 Web 服务（`insightclass serve`）

支持摄像头实时检测、视频上传检测、批量检测、摄像头管理和 Dashboard 统计。





启动命令：

```
# HTTP 模式（默认端口 8000）
insightclass serve --host 0.0.0.0 --port 8000

# HTTPS 模式（自动生成自签名证书，局域网摄像头需要）
insightclass serve --host 0.0.0.0 --port 8000 --https

# 指定实验目录
insightclass serve --experiments-root experiments --port 8000
```

四、CLI 命令完整列表

所有命令通过 `insightclass <子命令>` 或 `python -m insightclass <子命令>` 调用。

数据处理命令

子命令	功能	核心参数	调用模块
<code>create-manifest</code>	创建数据集清单 (视频级划分)	<code>--config</code> , <code>--output</code>	<code>data.manifest</code>
<code>extract-frames</code>	从视频中抽帧	<code>--manifest</code> , <code>--fps</code> , <code>--max-frames-per-video</code> , <code>--target-width</code> , <code>--seed</code>	<code>data.video_ops</code>
<code>inspect-yolo</code>	检查 YOLO 标注质量	<code>--dataset-root</code> , <code>--class-config</code> , <code>--output</code> , <code>--min-box-area</code>	<code>data.yolo</code>
<code>write-yolo-yaml</code>	生成 Ultralytics 数据集 YAML	<code>--dataset-root</code> , <code>--class-config</code> , <code>--output</code>	<code>data.yolo</code>

训练推理命令

子命令	功能	核心参数	调用模块
<code>train</code>	训练模型	<code>--config</code> (YAML)	<code>backends.factory</code> → <code>ultralytics_backend</code>
<code>validate</code>	验证模型	<code>--config</code> (YAML)	同上
<code>predict</code>	推理预测 (图片/视频)	<code>--config</code> (YAML)	同上
<code>render-first-frame</code>	可视化第一帧检测结果	<code>--config</code> (YAML)	<code>visualization.pipeline</code>

实验管理命令

子命令	功能	核心参数	调用模块
<code>compare-experiments</code>	导出实验对比 CSV	<code>--experiments-root</code> , <code>--output</code>	<code>evaluation.experiments</code>
<code>view-experiments</code>	启动实验结果查看 Web 界面	<code>--experiments-root</code> , <code>--port</code>	<code>web.experiment_viewer</code>

Web 服务命令

子命令	功能	核心参数	调用模块
<code>serve</code>	启动主 Web 服务（摄像头检测+Dashboard）	<code>--host</code> , <code>--port</code> , <code>--https</code> , <code>--experiments-root</code>	<code>web.server</code>
<code>demo</code>	启动统一 Demo（推理+实验查看）	<code>--experiments-root</code> , <code>--class-config</code> , <code>--port</code>	<code>web.demo</code>

五、Web API 端点完整列表

主 Web 服务（`serve`）API

页面路由

路径	方法	功能
<code>/</code>	GET	主页（摄像头检测界面）
<code>/dashboard</code>	GET	Dashboard 统计页面

实验管理 API

路径	方法	功能
<code>/api/experiments</code>	GET	获取所有实验列表 (ID、权重路径、类别、指标)
<code>/api/settings/default-model</code>	GET	获取当前默认模型路径
<code>/api/settings/default-model</code>	POST	设置默认模型路径 (body: <code>{"model": "..."} </code>)

摄像头管理 API

路径	方法	功能
<code>/api/cameras</code>	GET	获取摄像头列表 (内置 + 自定义)
<code>/api/cameras</code>	POST	添加自定义摄像头 (body: ip, name, username, password, port, note)
<code>/api/cameras/{ip}</code>	PUT	更新摄像头信息 (别名、凭据等)
<code>/api/cameras/{ip}</code>	DELETE	删除自定义摄像头
<code>/api/cameras/{ip}/test</code>	GET	测试单个摄像头连通性
<code>/api/cameras/test</code>	POST	批量测试摄像头连通性 (body: <code>{"cameras": [...] } </code>)

RTSP 流 API

路径	方法	功能
/api/stream/rtsp	GET	MJPEG 视频流 (query: rtsp_url) , 持久连接
/api/stream/stop	POST	停止当前 RTSP 流
/api/stream/status	GET	获取流状态 (idle / connecting / streaming / error)

检测 API

路径	方法	功能
/api/detect/frame	POST	单帧检测 (form: image 文件, model, confidence, iou)
/api/detect/upload	POST	视频上传检测 (form: video 文件, model, confidence, iou)
/api/detect/rtsp	POST	RTSP 摄像头单帧检测 (form: rtsp_url, model, confidence, iou)
/api/detect/batch-upload	POST	批量视频上传 (form: 多个 video 文件) , 返回 batch_id
/api/detect/batch/{batch_id}	POST	启动批量检测任务 (form: model, confidence, iou)
/api/detect/batch/{batch_id}	GET	查询批量任务状态
/api/detect/batch/{batch_id}/item/{index}	GET	获取批量任务中单个视频的 详细结果
/api/detect/batch/{batch_id}/export	GET	导出批量检测结果 (query: format=json/csv)

Dashboard API

路径	方法	功能
/api/dashboard/stats	GET	获取所有摄像头的检测统计数据
/api/dashboard/report	GET/POST	导出 Dashboard 统计报告 (CSV 下载)
/api/dashboard/history	GET	获取 24 小时历史数据 (用于图表展示, 当前为模拟数据)

Demo 应用 (demo) API

路径	方法	功能
/	GET	Demo 主页
/api/experiments	GET	获取实验列表
/api/experiments/{exp_id}/results.csv	GET	获取实验训练指标 CSV
/api/experiments/{exp_id}/confusion_matrix	GET	获取混淆矩阵图片
/api/experiments/{exp_id}/results.png	GET	获取训练结果图片
/api/detect/image	POST	图片检测 (返回标注后的 base64 图片 + 检测信息)

实验查看器 (`view-experiments`) API

路径	方法	功能
<code>/</code>	GET	实验查看主页
<code>/api/experiments</code>	GET	获取实验列表
<code>/api/experiments/{exp_id}/results.csv</code>	GET	获取实验训练指标 CSV
<code>/api/experiments/{exp_id}/confusion_matrix</code>	GET	获取混淆矩阵图片
<code>/api/experiments/{exp_id}/results.png</code>	GET	获取训练结果图片

六、核心模块详解

6.1 `cli.py` — 命令行入口

职责：接收用户输入的命令，分发到对应的处理模块。

工作方式：

- 使用 Python 标准库 `argparse` 定义 12 个子命令
- 每个子命令解析参数后，调用对应模块的功能
- 例如 `insightclass train --config xxx.yaml` 会：读取 YAML → 创建 TrainingConfig → 调用后端的 train 方法

6.2 `schemas.py` — 核心数据结构

职责：定义整个系统中流转的数据格式，就像"数据的骨架"。

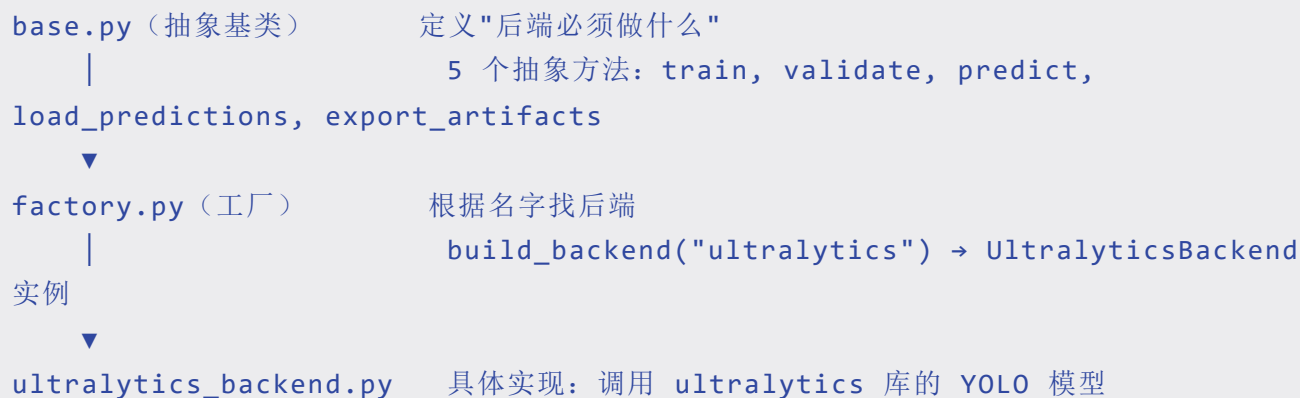
所有结构都用 `@dataclass(slots=True)` 定义（一种轻量级的类），每个都有 `to_dict()` 方法可以转成字典。

数据结构	用途	关键字段
<code>DatasetManifest</code>	描述一个数据集	<code>dataset_name</code> , <code>dataset_version</code> , <code>splits</code> (哪些视频分到 <code>train/val/test</code>) , <code>classes</code> , <code>display_names</code> , <code>video_metadata</code>
<code>TrainingConfig</code>	训练参数	<code>backend</code> , <code>task</code> , <code>data_config_path</code> , <code>model_weights</code> , <code>image_size</code> , <code>epochs</code> , <code>batch_size</code> , <code>device</code> , <code>project_dir</code> , <code>run_name</code> , <code>seed</code> , <code>patience</code> , <code>workers</code> , <code>extra_args</code>
<code>InferenceConfig</code>	推理参数	<code>backend</code> , <code>weights_path</code> , <code>source</code> , <code>output_dir</code> , <code>confidence</code> , <code>iou</code> , <code>image_size</code> , <code>device</code> , <code>save_frames</code> , <code>save_video</code>
<code>ExperimentRecord</code>	一次训练的完整记录	<code>experiment_id</code> , <code>backend</code> , <code>model_weights</code> , <code>data_version</code> , <code>class_names</code> , <code>hyperparameters</code> , <code>metrics</code> , <code>artifacts</code>
<code>DetectionRecord</code>	单个检测框	<code>xyxy</code> (坐标) , <code>confidence</code> (置信度) , <code>class_id</code> , <code>class_name</code>
<code>FramePrediction</code>	一帧的所有检测结果	<code>frame_index</code> , <code>source_path</code> , <code>detections</code> 列表

6.3 `backends/` — 模型后端（策略模式）

职责：封装所有与 AI 模型交互的逻辑。采用"策略模式"，可以轻松切换不同的模型框架。

设计思路：



为什么要这样设计？

如果以后想用别的模型框架（比如 `detectron2`、`mmdetection`），只需要写一个新的后端文件，在 `factory.py` 注册一下，其他代码完全不用改。

DetectorBackend 抽象基类定义的 5 个方法：

方法	做什么	输入	输出
<code>train()</code>	调用 YOLO 训练	TrainingConfig	ExperimentRecord
<code>validate()</code>	调用 YOLO 验证	TrainingConfig	metrics 字典
<code>predict_images_or_video()</code>	调用 YOLO 推理	InferenceConfig	输出目录路径
<code>load_predictions_as_sv_detections()</code>	推理并返回结构化结果	InferenceConfig	FramePrediction 列表
<code>export_artifacts()</code>	收集训练产物路径	实验目录	artifact 路径字典

6.4 data/ — 数据处理模块

职责：处理从原始视频到可训练数据集的全过程。

manifest.py — 数据集清单

功能：扫描视频目录 → 按比例随机划分 train/val/test → 生成清单 YAML

关键设计：视频级别划分（不是帧级别）。同一个视频的所有帧必须在同一个划分中，否则会导致数据泄露（模型在训练时看到了测试集的帧）。

build_split_map() 边界处理逻辑：

- 使用固定种子（默认 42）随机打乱视频列表
- `train_end = max(1, int(total * train_ratio))` — 至少 1 个视频分给 train
- `val_end` 计算时确保 val 集不为空（当 `total >= 3` 时至少 1 个）
- 剩余视频归入 test 集

输入：raw_videos/ 目录（里面有若干 .mp4 文件）

|

▼

discover_videos() --- 递归扫描所有视频文件

|

▼

build_split_map() --- 用固定种子随机打乱，按比例划分

|

▼

create_manifest() --- 组装成 DatasetManifest 对象

|

▼

输出：manifest.yaml（记录每个划分里有哪些视频）

video_ops.py — 视频抽帧

功能：读取清单，逐个打开视频，按指定 FPS 截图保存。

处理逻辑：

1. 根据清单找到每个划分下的视频

2. 用 OpenCV 打开视频，获取原生 FPS 和总帧数
3. 按 `fps` 参数计算帧间隔 (`sample_interval = round(native_fps / fps)`)
4. 生成候选帧索引列表
5. 如果设了 `max_frames_per_video`，从候选中均匀随机采样
6. 用 `capture.set(CAP_PROP_POS_FRAMES, target_idx)` 跳转到目标帧（避免解码全部帧）
7. 如果设了 `target_width`，把图片缩放到指定宽度（保持比例，仅缩不放）
8. 保存为 JPG，文件名格式：`{视频名}_f{帧号:06d}.jpg`
9. 写一个 `frame_index.csv` 记录每张图的来源和元数据

yolo.py — 标注质量检查

功能：扫描 YOLO 格式的标注文件，报告各种问题。

检查项：

- 有图片但没有对应标注文件
- 标注文件为空
- 类别 ID 超出范围
- 坐标超出 [0,1] 范围
- 检测框面积太小（可配置 `--min-box-area`，默认 0.0005）
- 各类别的数量分布

6.5 evaluation/ — 实验管理模块

职责：保存、加载、汇总训练实验的记录。

核心功能：

- `save_experiment_record()`：把一次训练的完整信息保存为 JSON
- `collect_experiment_records()`：扫描 experiments 目录，收集所有实验记录
- `export_experiment_summary()`：把所有实验的指标导出成 CSV，方便对比

实验记录包含什么：

- 实验 ID（目录名）
- 使用的后端和模型权重

- 数据集版本
- 类别名称
- 超参数（完整配置）
- 指标（mAP、precision、recall 等）
- 产物路径（权重文件、结果图等）

6.6 `visualization/` — 可视化模块

职责：在图片或视频帧上绘制检测框和标签。

依赖：`supervision` 库（一个专门做检测可视化的库）。

工作流程：

1. 读取图片/视频帧
2. 把 `FramePrediction` 转成 `supervision.Detections` 格式
3. 用 `BoxAnnotator` 画框
4. 用 `LabelAnnotator` 写标签（类别名 + 置信度）
5. 保存结果图片

6.7 `web/` — Web 前端模块

职责：提供浏览器端的实时检测界面和数据分析 Dashboard。

`server.py` — 主 Web 服务

这是最复杂的 Web 模块，包含以下核心组件：

RtspStreamManager — RTSP 流管理器

问题：RTSP 摄像头连接建立慢（几秒），不能每次检测都重新连接。

方案：维护一个持久的后台线程，持续从摄像头读取帧并缓存最新一帧。

`RtspStreamManager` 工作流程：

```
start(rtsp_url)
```

```

|
▼
启动后台线程 _capture_loop()
|
▼
└─> cv2.VideoCapture(rtsp_url)  -- 打开 RTSP 连接
|
|   ▼
|   cap.read()  -- 读取一帧
|   |
|   |--- 成功 → cv2.imencode(".jpg") → 缓存到 self._frame
|   |
|   |--- 失败 → fail_count++
|   |
|   |   |--- fail_count > 50 → 尝试重连（最多 3 次，每次等 2 秒）
|   |   |
|   |   |   |--- 重连成功 → 继续循环
|   |   |   |--- 重连失败 → status="error", 停止
|   |   |
|   |   |--- fail_count <= 50 → 等 0.1 秒后重试
|   |
|   └--- 循环直到 self._running = False

```

状态机： `idle` → `connecting` → `streaming` / `error`

DashboardStats — 统计追踪器

职责：在内存中追踪每个摄像头的行为检测计数。

数据结构：

```

{
  "10.8.14.36": {
    "stats": {"phone_use": 0, "talking": 0, "sleeping": 0, "standing":
0},
    "last_update": "2026-06-03T14:30:00"
  },
  ...
}

```

线程安全：所有读写操作都通过 `threading.Lock` 保护。

使用场景：每次 `/api/detect/rtsp` 检测完成后，自动调用 `_dashboard_stats.record(ip, class_name)` 累加计数。

摄像头配置管理

系统支持两类摄像头：

1. **内置摄像头**（硬编码在 `DEFAULT_CAMERA_GROUPS` 中）：
 - `front`（前视角）：7 个 IP
 - `rear`（后视角）：9 个 IP
 - 默认凭据：环境变量 `RTSP_USERNAME` / `RTSP_PASSWORD`，默认端口 554
2. **自定义摄像头**（存储在 `configs/cameras.yaml` 中）：
 - 通过 API 动态添加/修改/删除
 - 可覆盖内置摄像头的别名

RTSP URL 格式： `rtsp://{username}:{password}@{ip}:{port}/Streaming/Channels/101`

批量检测流程

```
POST /api/detect/batch-upload  -- 上传多个视频，返回 batch_id
|
▼
POST /api/detect/batch/{batch_id}  -- 启动后台检测线程
|
▼
后台线程 _batch_detect_worker()  -- 逐个视频推理
|   每完成一个视频，更新 item["status"] = "done"
|
▼
GET /api/detect/batch/{batch_id}  -- 前端轮询查询状态
|
▼
GET /api/detect/batch/{batch_id}/export?format=csv  -- 导出结果
```

自动清理：任务完成后 1 小时自动删除临时文件和任务记录。

模型加载策略

- 服务启动时，后台线程预加载默认模型（`preload_model`）
- 默认模型优先级：`app.yaml` 中配置 > `baseline_yolo11n_v2_e80-2` 实验 > 第一个可用实验
- 模型按权重路径缓存，同一路径只加载一次
- 权重路径安全校验：必须在 `experiments/` 目录下，只允许 `.pt` 文件

`demo.py` — 统一 Demo 应用

独立的 FastAPI 应用，整合了图片推理和实验结果查看功能。

特点：

- 使用 PIL/Pillow 绘制检测框（支持中文标签）
- 自动检测系统字体（微软雅黑、黑体、Noto CJK、文泉驿、PingFang）
- 推理结果返回标注后的 base64 图片 + 检测信息 JSON

`experiment_viewer.py` — 实验查看器

轻量级的实验结果浏览工具，只读取实验记录和产物，不做推理。

`model_cache.py` — 模型缓存

```
_model_cache: dict[str, object] = {} # 权重路径 → YOLO 实例

def get_model(weights_path: str):
    if path not in _model_cache:
        _model_cache[path] = YOLO(path)
    return _model_cache[path]
```

6.8 `utils/` — 工具函数

文件	功能
<code>paths.py</code>	<code>ensure_dir()</code> 创建目录; <code>is_video_file()</code> 判断是否视频文件
<code>serialization.py</code>	<code>load_yaml()</code> / <code>save_yaml()</code> / <code>load_json()</code> / <code>save_json()</code> 读写文件

6.9 `optional.py` — 可选依赖检测

职责：优雅处理可选依赖缺失的情况。

核心思想：核心包只依赖 `numpy` + `PyYAML` + `opencv-python`。ultralytics、supervision、fastapi 等都是可选的，用到时才检测。

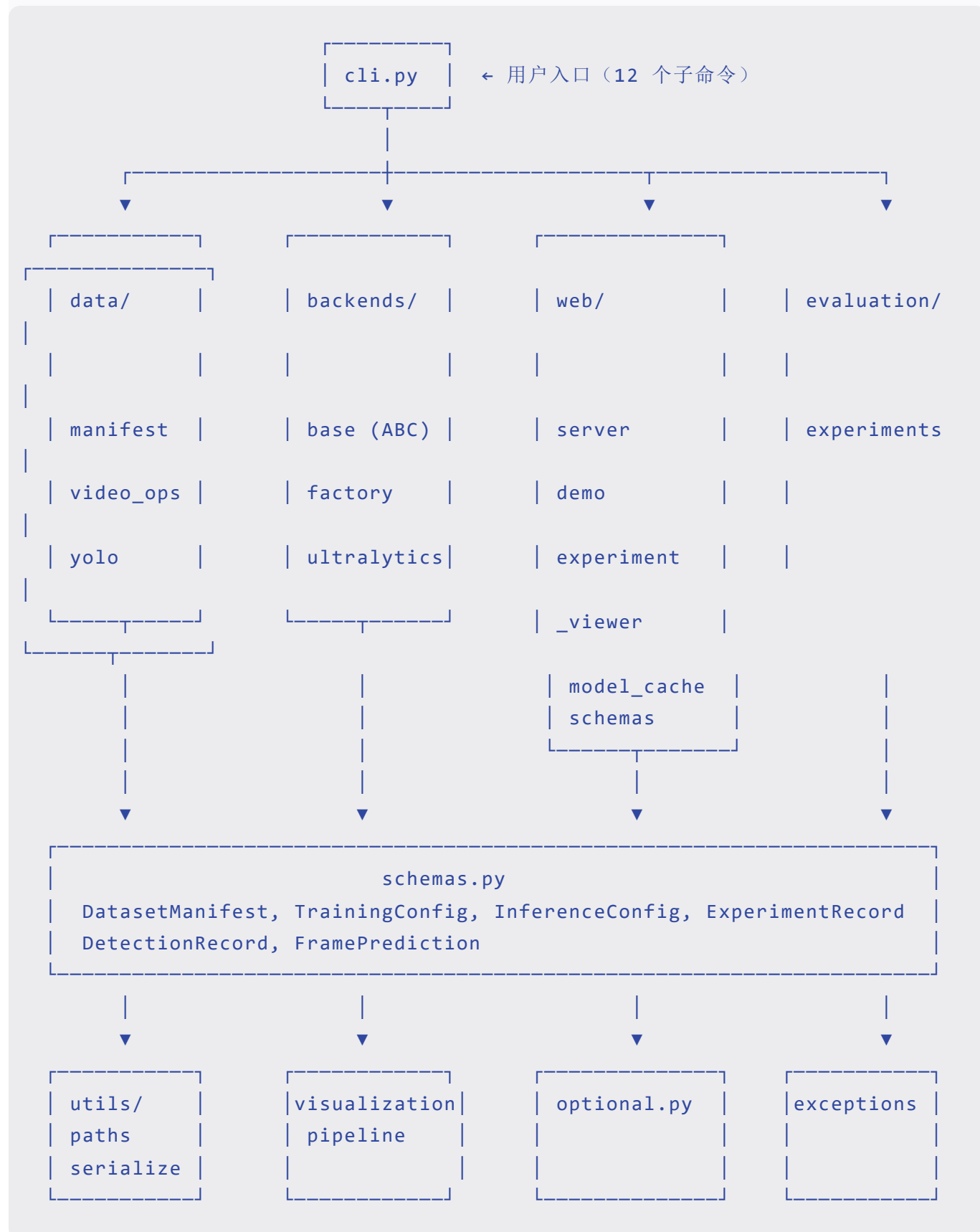
- `has_package("ultralytics")` → 返回 True/False
- `require_package("ultralytics", "训练功能")` → 如果没装就抛出友好错误信息

6.10 `scripts/` — 独立工具脚本

这些脚本不属于核心包，直接用 `python scripts/xxx.py` 运行。

脚本	功能
<code>record_multi_rtsp.py</code>	同时连接多路海康威视摄像头录制视频，支持子码流回退、主线程预览
<code>rtsp_preview.py</code>	单路摄像头预览+诊断（自动检测主码流/子码流哪个能用）
<code>sample_for_annotation.py</code>	从训练集随机抽样图片，方便拿去标注
<code>server_setup.sh</code>	Linux 服务器一键配置（创建 <code>venv</code> 、安装依赖、生成数据集配置）
<code>server_train.sh</code>	Linux 服务器一键启动训练（支持 <code>yolo11n/yolo26n</code> ，前台/后台模式）

七、模块调用关系图



基础层（被很多模块依赖，自己不依赖内部模块）：

- `schemas.py` — 数据结构

- `exceptions.py` — 异常类
- `utils/paths.py` — 路径工具
- `utils/serialization.py` — 序列化工具
- `optional.py` — 可选依赖检测

依赖方向：上层依赖下层，同层之间尽量不互相依赖（除了 `backends` 依赖 `evaluation` 和 `schemas`）。

八、配置文件说明

`configs/classes.yaml` — 类别定义（全局唯一）

```
classes:
- phone_use
- talking
- sleeping
- standing
display_names:
phone_use: 玩手机
talking: 交谈
sleeping: 打瞌睡
standing: 站立
```

这是整个项目的"类别字典"，所有模块都引用它。如果要新增行为类别，只改这一个文件是不够的，还需要重新标注和训练。

`configs/cameras.yaml` — 自定义摄像头配置（运行时生成）

```
cameras:
- ip: "10.8.14.100"
  name: "实验楼301"
  username: "admin"
  password: "1000phone"
  port: 554
  note: "自定义摄像头"
```

configs/app.yaml — 应用配置（运行时生成）

```
default_model: "experiments/baseline_yolo11n_v2_e80-2/weights/best.pt"
```

训练配置 YAML 结构

```
backend: ultralytics          # 使用哪个后端
task: detect                  # 任务类型
data_config_path: data/.../dataset.yaml # 数据集配置
model_weights: yolo11n.pt    # 预训练权重
image_size: 960              # 输入图片尺寸
epochs: 80                   # 训练轮数
batch_size: 16               # 批次大小
device: 0                    # GPU 编号（0 表示第一块 GPU）
project_dir: experiments     # 输出目录
run_name: baseline_yolo11n_v2_e80 # 实验名称
seed: 42                     # 随机种子（保证可复现）
patience: 30                # 早停耐心值
workers: 4                   # 数据加载线程数
extra_args:                  # YOLO 特有参数
  pretrained: true
  cache: false
  plots: true
```

九、技术亮点

9.1 可插拔后端设计

通过抽象基类 + 工厂模式，新增模型框架只需要：

1. 继承 `DetectorBackend`
2. 实现 5 个方法
3. 在 `factory.py` 注册

数据处理、评估、可视化代码完全不用改。

9.2 可选依赖懒加载

核心包只依赖 3 个轻量库。重量级依赖（ultralytics、supervision、fastapi）在真正用到时才 import，没装也不会报错（只在调用相关功能时提示）。

9.3 视频级数据划分

划分在视频级别而不是帧级别，防止同一个视频的不同帧同时出现在训练集和测试集（数据泄露）。

9.4 完整的实验管理

每次训练自动保存完整的实验记录（超参 + 指标 + 产物路径），支持跨实验对比导出 CSV。

9.5 RTSP 流持久连接

RtspStreamManager 用后台线程维护持久的 RTSP 连接，避免每次检测都重新建立连接（耗时数秒）。支持自动重连和状态监控。

9.6 模型缓存与预加载

YOLO 模型按权重路径缓存，服务启动时后台预加载默认模型，首次检测无需等待模型加载。

十、快速上手路径

如果你想...

目标	阅读文档	关键代码
了解项目全貌	本文档 + <code>docs/00_project_guide.md</code>	-
跑通数据处理	<code>docs/01_快速上手.md</code> + <code>docs/03_视频处理手册.md</code>	<code>data/manifest.py</code> , <code>data/video_ops.py</code>
用摄像头录数据	<code>docs/02_录制操作手册.md</code>	<code>scripts/record_multi_rtsp.py</code>
标注数据	<code>docs/04_X-AnyLabeling操作手册.md</code> + <code>docs/05_标注规范.md</code>	-
训练模型	<code>docs/06_实验手册.md</code> + <code>docs/07_服务器训练手册.md</code>	<code>backends/ultralytics_backend.py</code>
做推理和可视化	<code>docs/01_快速上手.md</code>	<code>backends/ultralytics_backend.py</code> , <code>visualization/pipeline.py</code>
搭建Web界面	<code>docs/09_前端使用手册.md</code>	<code>web/server.py</code> , <code>web/static/app.js</code>
查看实验结果	-	<code>web/experiment_viewer.py</code> , <code>web/demo.py</code>
添加新的模型后端	-	<code>backends/base.py</code> , <code>backends/factory.py</code>